cuVSLAM on Jetson AGX Orin

Installation and Real-Time Stereo + IMU Tracking (run_vio.py)

Target system

Component	Version
Board	NVIDIA Jetson AGX Orin Developer Kit (Tegra234)
L4T / JetPack	R39.2.0 -> JetPack 7.2 (Ubuntu 24.04)
CUDA	13.2
Python	3.12
Compute capability	8.7 (sm_87)
cuVSLAM	v17.0.0, built from source
Camera	Intel RealSense D455
pyrealsense2	2.58.4

> **Why a source build is required:** NVIDIA's prebuilt Orin wheels are cp310/cu12, built for JetPack 6.x. The aarch64 cu13/cp312-abi3 wheel is built for Jetson THOR, not Orin — the device families are not interchangeable. On JetPack 7.2 with CUDA 13.2 and Python 3.12 there is no matching prebuilt wheel, so cuVSLAM must be compiled from source.

Part 1 — Installation

1.1 Confirm the platform

```
cat /etc/nv_tegra_release       # R39 (release), REVISION: 2.0
nvcc --version                  # release 13.2
python3 --version               # 3.12.x
nvidia-smi --query-gpu=compute_cap --format=csv,noheader   # 8.7
df -h /                         # need ~10-15 GB free
```

1.2 Build dependencies

JetPack installs only the CUDA runtime libraries. The build needs the -dev packages, and the version suffix must match your CUDA version (13-2 here, not the 13-0 shown in NVIDIA's documentation).

```
sudo apt update
sudo apt install -y g++ cmake git git-lfs python3-dev python3-venv
apt-cache search libcublas-dev              # confirm available suffix
sudo apt install -y libcublas-dev-13-2 libcusolver-dev-13-2
```

If those names do not resolve:

```
sudo apt install -y cuda-libraries-dev-13-2
```

Enable the git-lfs hooks:

```
git lfs install
```

Check what is already present:

```
dpkg -l g++ cmake git git-lfs python3-dev python3-venv 2>/dev/null | grep '^ii'
```

CMake 3.28 (the Ubuntu 24.04 default) is sufficient; the requirement is 3.19 or newer.

1.3 Clone the repository

Skip the LFS blobs — they are example and test assets, not build inputs:

```
GIT_LFS_SKIP_SMUDGE=1 git clone https://github.com/nvidia-isaac/cuVSLAM.git ~/cuvslam
cd ~/cuvslam
```

1.4 Build the library

```
export CUVSLAM_SRC_DIR=~/cuvslam
export CUVSLAM_DST_DIR=~/cuvslam/build
./build_release.sh --cuda_arch=87
```

> **`--cuda_arch=87` matters:** This targets Orin only. Without it the build produces fatbins for every supported architecture, roughly quadrupling both build time and disk use.

1.5 Python bindings

```
python3 -m venv ~/cuvslam-env
source ~/cuvslam-env/bin/activate
pip install -U pip
CUVSLAM_BUILD_DIR=~/cuvslam/build pip install ~/cuvslam/python/
```

Verify:

```
python -c "import cuvslam; print(cuvslam.__version__)"      # -> 17.0.0
```

Check which odometry modes were compiled in:

```
python -c "import cuvslam; print([m for m in dir(cuvslam.Odometry.OdometryMode) if not m.startswith('_')])"
```

Expect Multicamera, Inertial, RGBD and Mono, plus Multisensor if cuNLS built successfully.

> **After every rebuild:** Re-run the pip install line above whenever libcuvslam.so is rebuilt. The bindings do not pick up a rebuilt library on their own.

A harmless warning may appear during install: "launch-ros 0.26.11 requires setuptools, which is not installed". This is unrelated to cuVSLAM — the venv can see system ROS packages. Run pip install setuptools to silence it.

1.6 Reclaim disk after a successful build

```
du -sh ~/cuvslam/build/_deps
rm -rf ~/cuvslam/build/_deps     # fetched dependency sources, safe to delete
```

Do not delete the rest of ~/cuvslam/build — the Python bindings link against libcuvslam.so there.

1.7 pyrealsense2 (required for the live examples)

```
source ~/cuvslam-env/bin/activate
python -c "import pyrealsense2 as rs; print(rs.__version__, rs.__file__)"
```

Expect 2.58.4 inside the venv. If it is missing:

```
pip install --timeout 120 pyrealsense2
```

If no aarch64/cp312 wheel exists, librealsense must be built from source with -DBUILD_PYTHON_BINDINGS=true and -DPYTHON_EXECUTABLE=$(which python3).

Part 2 — Real-Time Stereo + IMU (run_vio.py)

run_vio.py talks to the camera through pyrealsense2 directly. No ROS is involved. The tracker API is identical to the offline path; only the frame source differs.

> **Simpler than the offline path:** The live examples read the camera's factory-calibrated intrinsics, baseline and IMU extrinsics directly from the device, so there are no sensor.yaml files to write.

Available live examples in ~/cuvslam/examples/realsense/:

```
run_vio.py          stereo + IMU (visual-inertial)   <-- this one
run_stereo.py       stereo only
run_rgbd.py         mono + depth
run_multicamera.py  multiple stereo rigs
run_multisensor.py  requires cuNLS
camera_utils.py     camera configuration helpers
visualizer.py       rerun visualisation helpers
frame_agx_rig.yaml  rig config for AGX Orin
README.md
```

2.1 Set maximum performance

```
sudo nvpmodel -m 0       # MAXN (lifts the power cap)
sudo jetson_clocks       # pins clocks to max
sudo jetson_clocks --show | head -20
```

A correctly unlocked AGX Orin shows all 12 CPUs online at 2201600, the GPU at 1300500000 with 8 active TPCs, and EMC at 3199000000. MAXN alone is not enough — the governor still scales down without jetson_clocks.

2.2 Free the camera

Only one process can hold a RealSense device. Stop any ROS camera node, viewer, or leftover rs-* tool first:

```
ros2 node list | grep -i camera
pgrep -a -f "realsense|rs-"
```

Confirm the device is visible and free:

```
source ~/cuvslam-env/bin/activate
python -c "
import pyrealsense2 as rs
for d in rs.context().query_devices():
    print(d.get_info(rs.camera_info.name),
          d.get_info(rs.camera_info.firmware_version))
"
```

A busy or permission error means something still holds the device.

2.3 Check the IMU is enumerating

Inertial mode needs the camera's own IMU (the D455 carries a BMI085):

```
rs-enumerate-devices | grep -i -E "gyro|accel|motion"
```

A "Motion Module" section must appear in the stream profiles. If the device info line says "Imu Type: BMI085" but no Motion Module streams are listed, the IMU is present in hardware but not enumerating. Causes, cheapest first:

1. USB topology. The motion module enumerates on a separate interface from the UVC video streams and is more sensitive to marginal hubs and cables. Plug the camera directly into an Orin USB3 port. A "Physical Port" path containing .../2-3.4/... indicates a hub.
2. Firmware. Update with rs-fw-update -l, on a direct USB3 port, without interrupting it.
3. Kernel HID-sensor support. The D400 IMU is a HID device, separate from UVC, and JetPack kernels do not always ship these modules.

```
ls /sys/bus/iio/devices/
lsmod | grep -i hid_sensor
sudo modprobe hid-sensor-iio-common
sudo modprobe hid_sensor_accel_3d
sudo modprobe hid_sensor_gyro_3d
```

"module not found" means they were not built; building them against the R39 kernel source is a substantial detour.

2.4 Check the script before running

```
cd ~/cuvslam/examples/realsense
cat README.md
head -60 run_vio.py
```

a) IR emitter

The projected dot pattern helps the depth ASIC and wrecks stereo VSLAM. The dots are projected from the camera, move rigidly with it, and produce feature matches that carry no information about camera motion. Check whether the script disables it:

```
grep -i -n "emitter\|laser" run_vio.py camera_utils.py
```

If it does not, add this after starting the pipeline:

```
sensor = profile.get_device().first_depth_sensor()
sensor.set_option(rs.option.emitter_enabled, 0)
```

b) Rerun visualiser

```
grep -n "rr.init" run_vio.py
```

spawn=True fails over SSH with "winit EventLoopError: neither WAYLAND_DISPLAY nor ... nor DISPLAY is set". With no viewer attached the log queue then fills and the sender blocks: "Sender has been blocked for over 5 seconds waiting for space in channel".

In a real-time loop each block is wall-clock time during which the camera keeps producing frames — it causes dropped frames. Three options:

- Run from the Orin’s desktop, or use ssh -X, keeping spawn=True.
- Set spawn=False and add rr.save("run.rrd") to give it a real sink.
- Disable logging entirely for measurement runs.

2.5 Run

```
source ~/cuvslam-env/bin/activate
cd ~/cuvslam/examples/realsense
python run_vio.py
```

Quick Reference — Gotchas

Install

- No prebuilt Orin wheel for JetPack 7 / CUDA 13 / Python 3.12. Do not use the Thor cu13 wheel on Orin.
- Always pass --cuda_arch=87.
- Re-run pip install ~/cuvslam/python/ after every rebuild.

Environment

- cuvslam and pyrealsense2 live in the venv; ros2 lives in system Python. Activate the venv for cuvslam work, deactivate for ROS.
- sudo pip fails because sudo resets PATH. python3 -m pip3 is never valid — the module is always pip. System Python on Ubuntu 24.04 has no pip until sudo apt install python3-pip.

Camera

- IR emitter left on is the single most common cause of bad tracking. Verify it.
- infra1 and infra2 are monochrome by design — correct, not a fault. They are the stereo pair; depth is computed from them.
- Only one process may hold the device at a time.
- USB hubs cause dropped frames and can prevent the IMU enumerating.

Performance

- nvpmodel -m 0 and jetson_clocks, or the numbers are meaningless.
- Disable Rerun logging for timing runs — blocking invalidates the measurement and causes frame drops in real time.
- Sample at 100 ms and report percentiles, not a mean over 1-second windows.

IMU

- Only the camera's own IMU is valid for inertial mode. A separate robot-chassis IMU is not usable: different rigid body, unknown transform, independent clock. cuVSLAM's inertial mode assumes the IMU is rigidly mounted with the cameras and timestamped on the same clock.
- Stereo-only is a legitimate configuration, not a degraded fallback. On a differential-drive robot at walking pace the IMU contributes relatively little — it earns its keep under fast rotation and aggressive motion.